

Used model throughout: GPT-5 Thinking Mini

Prompt 1: "Implement a simple, beginner-friendly string right-justification helper. The function must be named `Utils::RightJustify`, take `const std::string& input` and `size_t width`, and return a `std::string`. Behavior: if `input.size() >= width` return `input` unchanged; otherwise return a new string containing `(width - input.size())` space characters followed by `input`. Use only simple loops and standard library types; keep code explicit and readable."

AI Response 1:

```
std::string Utils::RightJustify(const std::string& input, size_t width) {
    // If input is already as wide or wider than requested, return as-is.
    if (input.size() >= width) {
        return input;
    }

    // Compute how many spaces to add on the left.
    size_t pad = width - input.size();

    // Build a string of spaces.
    std::string result;
    result.reserve(width);
    for (size_t i = 0; i < pad; ++i) {
        result.push_back(' ');
    }

    // Append the original input after the padding.
    result += input;

    return result;
}
```

Prompt 2: "Implement a simple, beginner-friendly string left-justification helper. The function must be named `Utils::LeftJustify`, take `const std::string& input` and `size_t width`, and return a `std::string`. Behavior: if `input.size() >= width` return `input` unchanged; otherwise return a new string containing `input` followed by `(width - input.size())` space characters. Use only simple loops and standard library types; keep code explicit and readable."

AI Response 2:

```
std::string Utils::LeftJustify(const std::string& input, size_t width) {
    // If input is already as wide or wider than requested, return as-is.
    if (input.size() >= width) {
        return input;
    }

    // Compute how many spaces to add on the right.
    size_t pad = width - input.size();

    // Build the result: original input followed by pad spaces.
    std::string result;
    result.reserve(width);
    result += input;
    for (size_t i = 0; i < pad; ++i) {
```

```

        result.push_back(' ');
    }

    return result;
}

```

Prompt 3: "Implement `Utils::ContainsIgnoreCase(const std::string& source, const std::string& searchTerm)`. This must perform a case-insensitive substring search: return true if `searchTerm` (possibly shorter) appears anywhere inside `source` ignoring letter case. Use an explicit, beginner-friendly method: construct lowercase copies of both strings with `std::tolower` and perform `std::string::find` on them. Empty `searchTerm` should return true."

AI Response 3:

```

bool Utils::ContainsIgnoreCase(const std::string& source, const
std::string& searchTerm) {
    // Create lowercase copy of source.
    std::string lowerSource;
    lowerSource.reserve(source.size());
    for (size_t i = 0; i < source.size(); ++i) {
        unsigned char c = static_cast<unsigned char>(source[i]);
        lowerSource.push_back(static_cast<char>(std::tolower(c)));
    }

    // Create lowercase copy of searchTerm.
    std::string lowerTerm;
    lowerTerm.reserve(searchTerm.size());
    for (size_t i = 0; i < searchTerm.size(); ++i) {
        unsigned char c = static_cast<unsigned char>(searchTerm[i]);
        lowerTerm.push_back(static_cast<char>(std::tolower(c)));
    }

    // Empty search term matches everything by convention.
    if (lowerTerm.empty()) {
        return true;
    }

    // Use std::string::find on the lowercase versions.
    return lowerSource.find(lowerTerm) != std::string::npos;
}

```

Prompt 4: "Implement `Utils::PrintItemsTable(const std::vector<Item>& items)` to print a tabular list of items with dynamic column widths. The columns should be: ProductID (right-justified), Name (left), Category (left), UnitPrice (right, formatted with two decimals), Qty (right). Compute each column width as the maximum of the header length and the widest cell in that column. Use a simple fixed `COLUMN_SPACING` number of spaces between columns. Use `Utils::RightJustify` and `Utils::LeftJustify` helpers. Keep code straightforward and beginner-friendly."

AI Response 4:

```

void Utils::PrintItemsTable(const std::vector<Item>& items) {
    // Headers
    const std::string hdr1 = "ProductID";
    const std::string hdr2 = "Name";
}

```

```

const std::string hdr3 = "Category";
const std::string hdr4 = "UnitPrice";
const std::string hdr5 = "Qty";

// Start widths equal to header lengths.
size_t w1 = hdr1.size();
size_t w2 = hdr2.size();
size_t w3 = hdr3.size();
size_t w4 = hdr4.size();
size_t w5 = hdr5.size();

// Helper to format price with two decimal places.
auto FormatPrice = [](double price) -> std::string {
    std::ostringstream oss;
    oss.setf(std::ios::fixed);
    oss.precision(2);
    oss << price;
    return oss.str();
};

// Compute maximum widths based on data.
for (size_t i = 0; i < items.size(); ++i) {
    const Item& it = items[i];
    std::string s1 = std::to_string(it.GetProductID());
    if (s1.size() > w1) w1 = s1.size();

    const std::string& s2 = it.GetItemName();
    if (s2.size() > w2) w2 = s2.size();

    const std::string& s3 = it.GetCategory();
    if (s3.size() > w3) w3 = s3.size();

    std::string s4 = FormatPrice(it.GetUnitPrice());
    if (s4.size() > w4) w4 = s4.size();

    std::string s5 = std::to_string(it.GetQuantity());
    if (s5.size() > w5) w5 = s5.size();
}

// Spaces between columns (adjustable).
const size_t COLUMN_SPACING = 3;
std::string gap;
for (size_t i = 0; i < COLUMN_SPACING; ++i) gap.push_back(' ');

// Print header
std::string header = Utils::RightJustify(hdr1, w1) + gap
    + Utils::LeftJustify(hdr2, w2) + gap
    + Utils::LeftJustify(hdr3, w3) + gap
    + Utils::RightJustify(hdr4, w4) + gap
    + Utils::RightJustify(hdr5, w5);
std::cout << header << "\n";

// Print separator line
size_t sepLen = w1 + w2 + w3 + w4 + w5 + (COLUMN_SPACING * 4);
for (size_t i = 0; i < sepLen; ++i) std::cout.put('-');
std::cout << "\n";

// Print rows
for (size_t i = 0; i < items.size(); ++i) {
    const Item& it = items[i];
    std::string col1 = std::to_string(it.GetProductID());

```

```

        std::string col2 = it.GetItemName();
        std::string col3 = it.GetCategory();

        std::ostringstream oss;
        oss.setf(std::ios::fixed);
        oss.precision(2);
        oss << it.GetUnitPrice();
        std::string col4 = oss.str();

        std::string col5 = std::to_string(it.GetQuantity());

        std::string line = Utils::RightJustify(col1, w1) + gap
                        + Utils::LeftJustify(col2, w2) + gap
                        + Utils::LeftJustify(col3, w3) + gap
                        + Utils::RightJustify(col4, w4) + gap
                        + Utils::RightJustify(col5, w5);
        std::cout << line << "\n";
    }
}

```

Prompt 5: "Implement `AdminActions::ShowAllUsers(const std::vector<User>& users) const`. The function must compute dynamic column widths for the columns UserID (right), Password (left), Name (left), Address (left), Contact (right). It must produce a header row, a separator line sized to the full table width, then one row per user using `Utils::RightJustify` and `Utils::LeftJustify`. Keep the logic explicit and easy to follow."

AI Response 5:

```

void AdminActions::ShowAllUsers(const std::vector<User>& users) const {
    // Column headers
    std::string hdr1 = "UserID";
    std::string hdr2 = "Password";
    std::string hdr3 = "Name";
    std::string hdr4 = "Address";
    std::string hdr5 = "Contact";

    // Start widths as header lengths
    size_t w1 = hdr1.size();
    size_t w2 = hdr2.size();
    size_t w3 = hdr3.size();
    size_t w4 = hdr4.size();
    size_t w5 = hdr5.size();

    // Find maximum widths from data
    for (size_t i = 0; i < users.size(); ++i) {
        const User& u = users[i];

        std::string s1 = std::to_string(u.GetUserID());
        if (s1.size() > w1) w1 = s1.size();

        if (u.GetPassword().size() > w2) w2 = u.GetPassword().size();
        if (u.GetName().size() > w3) w3 = u.GetName().size();
        if (u.GetAddress().size() > w4) w4 = u.GetAddress().size();

        std::string s5 = std::to_string(u.GetContactNumber());
        if (s5.size() > w5) w5 = s5.size();
    }
}

```

```

// Column spacing
const size_t COLUMN_SPACING = 3;
std::string gap;
for (size_t i = 0; i < COLUMN_SPACING; ++i) gap.push_back(' ');

// Print header
std::cout << Utils::RightJustify(hdr1, w1) << gap
           << Utils::LeftJustify(hdr2, w2) << gap
           << Utils::LeftJustify(hdr3, w3) << gap
           << Utils::LeftJustify(hdr4, w4) << gap
           << Utils::RightJustify(hdr5, w5) << "\n";

// Separator
size_t sepLen = w1 + w2 + w3 + w4 + w5 + (COLUMN_SPACING * 4);
for (size_t i = 0; i < sepLen; ++i) std::cout.put('-');
std::cout << "\n";

// Print rows
for (size_t i = 0; i < users.size(); ++i) {
    const User& u = users[i];
    std::cout << Utils::RightJustify(std::to_string(u.GetUserID()), w1)
<< gap
           << Utils::LeftJustify(u.GetPassword(), w2) << gap
           << Utils::LeftJustify(u.GetName(), w3) << gap
           << Utils::LeftJustify(u.GetAddress(), w4) << gap
           <<
    Utils::RightJustify(std::to_string(u.GetContactNumber()), w5) << "\n";
}
}

```

Prompt 6: "Implement `AdminActions::ShowAllSellers(const std::vector<User>& users, const std::vector<Item>& items) const`. Behavior: compute number of items per seller from `items`, then print a table for sellers only (users who are sellers) with columns SellerID (right), Password (left), Name (left), Address (left), Contact (right), NumItems (right). Use dynamic widths as in previous functions and print rows in the same order users appear in the `users` vector when they are sellers."

AI Response 6:

```

void AdminActions::ShowAllSellers(const std::vector<User>& users, const
std::vector<Item>& items) const {
    // Count items per seller
    std::map<int,int> sellerCounts;
    for (size_t i = 0; i < items.size(); ++i) {
        int sid = items[i].GetSellerID();
        if (sellerCounts.find(sid) == sellerCounts.end()) sellerCounts[sid]
= 1;
        else sellerCounts[sid] = sellerCounts[sid] + 1;
    }

    // Headers
    std::string hdr1 = "SellerID";
    std::string hdr2 = "Password";
    std::string hdr3 = "Name";
    std::string hdr4 = "Address";
    std::string hdr5 = "Contact";
    std::string hdr6 = "NumItems";

    // Start widths

```

```

size_t w1 = hdr1.size();
size_t w2 = hdr2.size();
size_t w3 = hdr3.size();
size_t w4 = hdr4.size();
size_t w5 = hdr5.size();
size_t w6 = hdr6.size();

// Prepare list of seller User pointers and compute widths
std::vector<const User*> sellers;
for (size_t i = 0; i < users.size(); ++i) {
    const User& u = users[i];
    int uid = u.GetUserID();
    if (sellerCounts.find(uid) != sellerCounts.end()) {
        sellers.push_back(&u);
        std::string s1 = std::to_string(uid);
        if (s1.size() > w1) w1 = s1.size();
        if (u.GetPassword().size() > w2) w2 = u.GetPassword().size();
        if (u.GetName().size() > w3) w3 = u.GetName().size();
        if (u.GetAddress().size() > w4) w4 = u.GetAddress().size();
        std::string s5 = std::to_string(u.GetContactNumber());
        if (s5.size() > w5) w5 = s5.size();
        std::string s6 = std::to_string(sellerCounts[uid]);
        if (s6.size() > w6) w6 = s6.size();
    }
}

// Column spacing
const size_t COLUMN_SPACING = 3;
std::string gap;
for (size_t i = 0; i < COLUMN_SPACING; ++i) gap.push_back(' ');

// Print header
std::cout << Utils::RightJustify(hdr1, w1) << gap
           << Utils::LeftJustify(hdr2, w2) << gap
           << Utils::LeftJustify(hdr3, w3) << gap
           << Utils::LeftJustify(hdr4, w4) << gap
           << Utils::RightJustify(hdr5, w5) << gap
           << Utils::RightJustify(hdr6, w6) << "\n";

// Separator
size_t sepLen = w1 + w2 + w3 + w4 + w5 + w6 + (COLUMN_SPACING * 5);
for (size_t i = 0; i < sepLen; ++i) std::cout.put('-');
std::cout << "\n";

// Print rows for each seller in original users order
for (size_t i = 0; i < sellers.size(); ++i) {
    const User* u = sellers[i];
    int uid = u->GetUserID();
    std::cout << Utils::RightJustify(std::to_string(uid), w1) << gap
              << Utils::LeftJustify(u->GetPassword(), w2) << gap
              << Utils::LeftJustify(u->GetName(), w3) << gap
              << Utils::LeftJustify(u->GetAddress(), w4) << gap
              << Utils::RightJustify(std::to_string(u-
>GetContactNumber()), w5) << gap
              << Utils::RightJustify(std::to_string(sellerCounts[uid]),
w6) << "\n";
    }
}

```

Prompt 7: "Implement `AdminActions::ShowSellersSales(const std::vector<Transaction>& transactions, int startMonth, int startDay, int`

startYear, int endMonth, int endDay, int endYear) const. The function should aggregate total sales per seller for transactions whose date is within the inclusive date range and then print a two-column table SellerID (right) and TotalSales (right) with dynamic widths. Use an explicit date-in-range helper in the function body. Keep code simple and readable."

AI Response 7:

```
void AdminActions::ShowSellersSales(const std::vector<Transaction>&
transactions,
                                int startMonth, int startDay, int
startYear,
                                int endMonth, int endDay, int endYear)
const {
    // Helper to check inclusive date range
    auto DateInRange = [](const Date& d, int sm, int sd, int sy, int em,
int ed, int ey) -> bool {
        if (d.year < sy) return false;
        if (d.year > ey) return false;
        if (d.year == sy) {
            if (d.month < sm) return false;
            if (d.month == sm && d.day < sd) return false;
        }
        if (d.year == ey) {
            if (d.month > em) return false;
            if (d.month == em && d.day > ed) return false;
        }
        return true;
    };

    // Aggregate totals per seller
    std::map<int,double> totals;
    for (size_t i = 0; i < transactions.size(); ++i) {
        const Transaction& t = transactions[i];
        if (DateInRange(t.GetDate(), startMonth, startDay, startYear,
endMonth, endDay, endYear)) {
            int sid = t.GetSellerID();
            totals[sid] = totals[sid] + t.GetTotalAmount();
        }
    }

    // Headers
    std::string hdr1 = "SellerID";
    std::string hdr2 = "TotalSales";
    size_t w1 = hdr1.size();
    size_t w2 = hdr2.size();

    // Compute widths
    for (std::map<int,double>::const_iterator it = totals.begin(); it !=
totals.end(); ++it) {
        std::string s1 = std::to_string(it->first);
        if (s1.size() > w1) w1 = s1.size();
        std::ostringstream oss;
        oss.setf(std::ios::fixed);
        oss.precision(2);
        oss << it->second;
        std::string s2 = oss.str();
        if (s2.size() > w2) w2 = s2.size();
    }
```

```

// Spacing
const size_t COLUMN_SPACING = 3;
std::string gap;
for (size_t i = 0; i < COLUMN_SPACING; ++i) gap.push_back(' ');

// Print header
std::cout << Utils::RightJustify(hdr1, w1) << gap <<
Utils::RightJustify(hdr2, w2) << "\n";

// Separator
for (size_t i = 0; i < (w1 + w2 + COLUMN_SPACING); ++i)
std::cout.put('-');
std::cout << "\n";

// Print rows sorted by sellerID (map iteration order)
for (std::map<int,double>::const_iterator it = totals.begin(); it !=
totals.end(); ++it) {
    std::ostringstream oss;
    oss.setf(std::ios::fixed);
    oss.precision(2);
    oss << it->second;
    std::cout << Utils::RightJustify(std::to_string(it->first), w1) <<
gap
                                << Utils::RightJustify(oss.str(), w2) << "\n";
}
}

```

Prompt 8: "Implement `AdminActions::ShowShopaholics(const std::vector<Transaction>& transactions, int startMonth, int startDay, int startYear, int endMonth, int endDay, int endYear) const`. Aggregate total amount bought per buyer in the inclusive date range and print a two-column table BuyerID (right) and TotalBought (right) with dynamic widths. Keep the implementation simple and explicit."

AI Response 8:

```

void AdminActions::ShowShopaholics(const std::vector<Transaction>&
transactions,
                                int startMonth, int startDay, int
startYear,
                                int endMonth, int endDay, int endYear)
const {
    // Helper to check inclusive date range
    auto DateInRange = [](const Date& d, int sm, int sd, int sy, int em,
int ed, int ey) -> bool {
        if (d.year < sy) return false;
        if (d.year > ey) return false;
        if (d.year == sy) {
            if (d.month < sm) return false;
            if (d.month == sm && d.day < sd) return false;
        }
        if (d.year == ey) {
            if (d.month > em) return false;
            if (d.month == em && d.day > ed) return false;
        }
        return true;
    };

    // Aggregate totals per buyer

```



```

        std::map<int,double> totals;
        for (size_t i = 0; i < transactions.size(); ++i) {
            const Transaction& t = transactions[i];
            if (DateInRange(t.GetDate(), startMonth, startDay, startYear,
endMonth, endDay, endYear)) {
                int bid = t.GetBuyerID();
                totals[bid] = totals[bid] + t.GetTotalAmount();
            }
        }

        // Headers
        std::string hdr1 = "BuyerID";
        std::string hdr2 = "TotalBought";
        size_t w1 = hdr1.size();
        size_t w2 = hdr2.size();

        // Compute widths
        for (std::map<int,double>::const_iterator it = totals.begin(); it !=
totals.end(); ++it) {
            std::string s1 = std::to_string(it->first);
            if (s1.size() > w1) w1 = s1.size();
            std::ostringstream oss;
            oss.setf(std::ios::fixed);
            oss.precision(2);
            oss << it->second;
            std::string s2 = oss.str();
            if (s2.size() > w2) w2 = s2.size();
        }

        // Spacing
        const size_t COLUMN_SPACING = 3;
        std::string gap;
        for (size_t i = 0; i < COLUMN_SPACING; ++i) gap.push_back(' ');

        // Print header
        std::cout << Utils::RightJustify(hdr1, w1) << gap <<
Utils::RightJustify(hdr2, w2) << "\n";

        // Separator
        for (size_t i = 0; i < (w1 + w2 + COLUMN_SPACING); ++i)
std::cout.put('-');
        std::cout << "\n";

        // Print rows
        for (std::map<int,double>::const_iterator it = totals.begin(); it !=
totals.end(); ++it) {
            std::ostringstream oss;
            oss.setf(std::ios::fixed);
            oss.precision(2);
            oss << it->second;
            std::cout << Utils::RightJustify(std::to_string(it->first), w1) <<
gap
                                << Utils::RightJustify(oss.str(), w2) << "\n";
        }
    }
}

```

Prompt 9: "Implement `bool ShoppingApp::RegisterUser(const User& user)` that enforces the project constraint of at most 100 users and unique userID. If the limit is reached return false and print a clear diagnostic message to `std::cerr` stating `RegisterUser failed: user limit reached (100)`.. If a duplicate userID is found, return false and print

RegisterUser failed: duplicate userID found: <id>. On success add the user to the app's user container and return true. Keep the code simple and straightforward."

AI Response 9:

```
bool ShoppingApp::RegisterUser(const User& user) {
    // Enforce maximum of 100 users.
    if ((int)users_.size() >= 100) {
        std::cerr << "RegisterUser failed: user limit reached (100)." <<
std::endl;
        return false;
    }

    // Check for duplicate userID.
    for (size_t i = 0; i < users_.size(); ++i) {
        if (users_[i].GetUserID() == user.GetUserID()) {
            std::cerr << "RegisterUser failed: duplicate userID found: " <<
user.GetUserID() << std::endl;
            return false;
        }
    }

    // Add user and succeed.
    users_.push_back(user);
    return true;
}
```

Prompt 10: "Provide beginner-friendly implementations for User model functions: default constructor, overloaded constructor, getters for userID, password, name, address, contactNumber, and setters for those fields. Keep code explicit and avoid advanced syntax."

AI Response 10:

```
User::User()
    : userID_(0), password_(), name_(), address_(), contactNumber_(0) {
}

User::User(int userID, const std::string& password, const std::string&
name, const std::string& address, long long contactNumber)
    : userID_(userID), password_(password), name_(name), address_(address),
contactNumber_(contactNumber) {
}

int User::GetUserID() const { return userID_; }
const std::string& User::GetPassword() const { return password_; }
const std::string& User::GetName() const { return name_; }
const std::string& User::GetAddress() const { return address_; }
long long User::GetContactNumber() const { return contactNumber_; }

void User::SetUserID(int id) { userID_ = id; }
void User::SetPassword(const std::string& password) { password_ = password;
}
void User::SetName(const std::string& name) { name_ = name; }
void User::SetAddress(const std::string& address) { address_ = address; }
void User::SetContactNumber(long long number) { contactNumber_ = number; }
```

Prompt 11: "Provide beginner-friendly implementations for `Item` model functions: default constructor, overloaded constructor, getters for `productID`, `itemName`, `category`, `description`, `quantity`, `unitPrice`, `sellerID`, and setters for those fields. Keep code explicit."

AI Response 11:

```
Item::Item()
    : productID_(0), itemName_(), category_(), description_(),
    quantity_(0), unitPrice_(0.0), sellerID_(0) {
}

Item::Item(int productID, const std::string& itemName, const std::string&
category,
    const std::string& description, int quantity, double unitPrice,
int sellerID)
    : productID_(productID), itemName_(itemName), category_(category),
    description_(description),
    quantity_(quantity), unitPrice_(unitPrice), sellerID_(sellerID) {
}

int Item::GetProductID() const { return productID_; }
const std::string& Item::GetItemName() const { return itemName_; }
const std::string& Item::GetCategory() const { return category_; }
const std::string& Item::GetDescription() const { return description_; }
int Item::GetQuantity() const { return quantity_; }
double Item::GetUnitPrice() const { return unitPrice_; }
int Item::GetSellerID() const { return sellerID_; }

void Item::SetProductID(int id) { productID_ = id; }
void Item::SetItemName(const std::string& name) { itemName_ = name; }
void Item::SetCategory(const std::string& category) { category_ = category; }
void Item::SetDescription(const std::string& desc) { description_ = desc; }
void Item::SetQuantity(int qty) { quantity_ = qty; }
void Item::SetUnitPrice(double price) { unitPrice_ = price; }
void Item::SetSellerID(int id) { sellerID_ = id; }
```

Prompt 12: "Provide beginner-friendly implementations for `Cart` and `CartItem` functions: `CartItem` default and overloaded constructors, `Cart` default and vector-based constructors, `GetItems()`, `AddItem(const CartItem& item)` (merge quantities if product exists; enforce max 10 distinct items), `RemoveItemByProductId(int productId)`, `RemoveItemsBySellerId(int sellerId)`, `UpdateQuantity(int productId, int newQuantity)` (remove if `newQuantity <= 0`), `Clear()`, and `Size()`. Use simple loops and vector operations only."

AI Response 12:

```
CartItem::CartItem()
    : productID(0), sellerID(0), quantity(0), unitPrice(0.0), itemName() {
}

CartItem::CartItem(int productID_, int sellerID_, int quantity_, double
unitPrice_, const std::string& itemName_)
    : productID(productID_), sellerID(sellerID_), quantity(quantity_),
    unitPrice(unitPrice_), itemName(itemName_) {
}
```

```

Cart::Cart()
    : items_() {
}

Cart::Cart(const std::vector<CartItem>& items)
    : items_(items) {
}

const std::vector<CartItem>& Cart::GetItems() const {
    return items_;
}

bool Cart::AddItem(const CartItem& item) {
    // If product already in cart, aggregate quantity.
    for (size_t i = 0; i < items_.size(); ++i) {
        if (items_[i].productID == item.productID) {
            items_[i].quantity += item.quantity;
            return true;
        }
    }

    // Enforce maximum of 10 distinct items.
    if ((int)items_.size() >= 10) {
        return false;
    }

    // Otherwise add as new distinct item.
    items_.push_back(item);
    return true;
}

bool Cart::RemoveItemByProductId(int productId) {
    for (size_t i = 0; i < items_.size(); ++i) {
        if (items_[i].productID == productId) {
            items_.erase(items_.begin() + i);
            return true;
        }
    }
    return false;
}

bool Cart::RemoveItemsBySellerId(int sellerId) {
    bool removed = false;
    for (size_t i = 0; i < items_.size(); ) {
        if (items_[i].sellerID == sellerId) {
            items_.erase(items_.begin() + i);
            removed = true;
        } else {
            ++i;
        }
    }
    return removed;
}

bool Cart::UpdateQuantity(int productId, int newQuantity) {
    for (size_t i = 0; i < items_.size(); ++i) {
        if (items_[i].productID == productId) {
            if (newQuantity <= 0) {
                items_.erase(items_.begin() + i);
                return true;
            }
        }
    }
}

```

```

        items_[i].quantity = newQuantity;
        return true;
    }
    return false;
}

void Cart::Clear() {
    items_.clear();
}

int Cart::Size() const {
    return static_cast<int>(items_.size());
}

```

Prompt 13: "Provide beginner-friendly implementations for transaction-related model functions: Date constructors, TransactionItem default and overloaded constructors, Transaction default and overloaded constructors, getters for date/items/buyerID/sellerID/totalAmount, setters for these fields, and AddItem(const TransactionItem& item) which refuses to add more than 5 items and automatically updates totalAmount_. Keep implementation explicit."

AI Response 13:

```

Date::Date()
    : month(0), day(0), year(0) {
}

Date::Date(int m, int d, int y)
    : month(m), day(d), year(y) {
}

TransactionItem::TransactionItem()
    : productID(0), itemName(), quantity(0), unitPrice(0.0) {
}

TransactionItem::TransactionItem(int productID_, const std::string&
itemName_, int quantity_, double unitPrice_)
    : productID(productID_), itemName(itemName_), quantity(quantity_),
unitPrice(unitPrice_) {
}

Transaction::Transaction()
    : date_(), items_(), buyerID_(0), sellerID_(0), totalAmount_(0.0) {
}

Transaction::Transaction(const Date& date, const
std::vector<TransactionItem>& items, int buyerID, int sellerID, double
totalAmount)
    : date_(date), items_(items), buyerID_(buyerID), sellerID_(sellerID),
totalAmount_(totalAmount) {
}

const Date& Transaction::GetDate() const { return date_; }
const std::vector<TransactionItem>& Transaction::GetItems() const { return
items_; }
int Transaction::GetBuyerID() const { return buyerID_; }
int Transaction::GetSellerID() const { return sellerID_; }
double Transaction::GetTotalAmount() const { return totalAmount_; }

```

```

void Transaction::SetDate(const Date& date) { date_ = date; }
void Transaction::SetItems(const std::vector<TransactionItem>& items) {
    items_ = items; }
void Transaction::SetBuyerID(int id) { buyerID_ = id; }
void Transaction::SetSellerID(int id) { sellerID_ = id; }
void Transaction::SetTotalAmount(double amount) { totalAmount_ = amount; }

bool Transaction::AddItem(const TransactionItem& item) {
    // Enforce at most 5 items in a transaction.
    if ((int)items_.size() >= 5) {
        return false;
    }

    items_.push_back(item);

    // Update total amount (unitPrice * quantity).
    totalAmount_ += item.unitPrice * static_cast<double>(item.quantity);
    return true;
}

```

Prompt 14: "Implement the user-menu greeting flow function `void UserMenu::Show()` used during login. Behavior: prompt the user for numeric user ID (validate input), then read password as a line. Use `userActions_.Login(app_.GetUsers(), userId, password)` to authenticate (assume this exists). On successful login find the user's name from `app_.GetUsers()` and display a greeting line `Welcome, <username>!` at top of the user menu. Then loop and present the menu options 1) Sell Menu 2) Buy Menu 3) Exit User Menu; read numeric choice and call the corresponding submenus (`SellerMenu/BuyerMenu`) when chosen; on exit print a goodbye message containing the user's name. Keep control flow clear and beginner-friendly."

AI Response 14:

```

void UserMenu::Show() {
    std::cout << "--- User Login ---\n";

    int userId = 0;
    std::cout << "Enter user ID: ";
    if (!(std::cin >> userId)) {
        std::cin.clear();
        std::string junk;
        std::getline(std::cin, junk);
        std::cout << "Invalid user ID.\n";
        return;
    }

    // Consume leftover newline before reading password line.
    std::string tmp;
    std::getline(std::cin, tmp);

    std::string password;
    std::cout << "Enter password: ";
    std::getline(std::cin, password);

    // Authenticate using helper action (assumed to exist).
    bool ok = userActions_.Login(app_.GetUsers(), userId, password);
    if (!ok) {
        std::cout << "Login failed. Returning to main menu.\n";
    }
}

```

```

        return;
    }

    // Find logged-in user's name for greeting.
    std::string loggedInUserName = "User";
    const std::vector<User>& allUsers = app_.GetUsers();
    for (size_t i = 0; i < allUsers.size(); ++i) {
        if (allUsers[i].GetUserID() == userId) {
            loggedInUserName = allUsers[i].GetName();
            break;
        }
    }

    // Main user menu loop with greeting.
    while (true) {
        std::cout << "\n=== USER MENU (ID " << userId << ") ===\n";
        std::cout << "Welcome, " << loggedInUserName << "!\n\n";

        std::cout << "1. Sell Menu\n";
        std::cout << "2. Buy Menu\n";
        std::cout << "3. Exit User Menu\n";
        std::cout << "Choose option: ";

        int choice = 0;
        if (!(std::cin >> choice)) {
            std::cin.clear();
            std::string junk;
            std::getline(std::cin, junk);
            std::cout << "Invalid input.\n";
            continue;
        }

        // Consume newline after numeric choice
        std::getline(std::cin, tmp);

        if (choice == 1) {
            SellerMenu sm(app_, userId);
            sm.Show();
        } else if (choice == 2) {
            BuyerMenu bm(app_, userId);
            bm.Show();
        } else if (choice == 3) {
            std::cout << "Goodbye, " << loggedInUserName << ". Returning to
main menu.\n";
            break;
        } else {
            std::cout << "Unknown option.\n";
        }
    }
}

```